

👉 XAI Lecture 05

Rule Based Approaches

FAMAF - UNC

First Semester 2026



eXplainable
Artificial Intelligence

Disclaimer ---

This course is based on

Explainable Artificial Intelligence

From Simple Predictors to Complex Generative Models

Spring 2023, Harvard University

<https://interpretable-ml-class.github.io/>

The Annals of Applied Statistics
2015, Vol. 9, No. 3, 1350–1371
DOI: [10.1214/15-AOAS848](https://doi.org/10.1214/15-AOAS848)
© Institute of Mathematical Statistics, 2015

INTERPRETABLE CLASSIFIERS USING RULES AND BAYESIAN ANALYSIS: BUILDING A BETTER STROKE PREDICTION MODEL

BY BENJAMIN LETHAM^{*,1}, CYNTHIA RUDIN^{*,1}, TYLER H. MCCORMICK^{†,2}
AND DAVID MADIGAN^{‡,3}

Massachusetts Institute of Technology^{}, University of Washington[†]
and Columbia University[‡]*

We aim to produce predictive models that are not only accurate, but are also interpretable to human experts. Our models are decision lists, which consist of a series of *if...then...* statements (e.g., *if high blood pressure, then stroke*) that discretize a high-dimensional,

(Letham et al. 2015)

Contributions ---

- **Bayesian Rule Lists (BRL):** a generative model that outputs an ordered `if-then-else` decision list
- **Novel sparsity-inducing prior:** truncated Poisson priors over list length and rule complexity encourage short lists with simple conditions
- **Predictive accuracy on par** with Random Forests, SVMs, and CHADS₂ — while remaining fully interpretable

Decision List: Titanic Example ---

if male **and** adult **then** *survival probability* 21% (19%–23%)
else if 3rd class **then** *survival probability* 44% (38%–51%)
else if 1st class **then** *survival probability* 96% (92%–99%)
else *survival probability* 88% (82%–94%)

- **Rules fire in order** — the first matching rule determines the prediction
- Values in parentheses are **95% credible intervals** — the narrower, the more data behind that rule
- BRL doesn't output a single list — it maintains a **posterior distribution over many possible lists**

How Does BRL Work? ---

Mine candidate rules $\xrightarrow{\text{FP-Growth}}$ Define prior $\rightarrow$ Compute likelihood $\xrightarrow{\text{MCMC}}$
Posterior over lists

- **Step 1 — Mine:** extract frequent **if** conditions from data using FP-Growth
- **Step 2 — Prior:** favor short lists with simple rules (truncated Poisson)
- **Step 3 — Likelihood:** measure how well a list explains the observed labels
- **Step 4 — MCMC:** sample decision lists proportionally to their posterior probability

🍷 Pre-mined Rules ---



Minimum Support = 3

- FP-Growth extracts all frequent conditions (antecedents) from the data — e.g., {diabetes}, {age > 60}, {diabetes, age > 60}
- BRL only searches over this **pre-mined set** — this keeps the problem tractable and improves generalization

🍷 Bayesian Association Rules ---

A **Bayesian association rule** predicts a probability distribution over classes, not a single label:

$$a \rightarrow y \sim \text{Multinomial}(\theta_j), \quad \theta_j \sim \text{Dirichlet}(\alpha + \mathbf{N}_j)$$

In plain English: for each rule a , we estimate the probability of each class by combining prior belief (α) with observed counts ($\mathbf{N}_j$) — the more data, the more the prior is washed out.

Example: rule `age > 60 AND diabetes` captures 10 patients: 7 stroke, 3 no stroke. With $\alpha = (1, 1)$, posterior = $\text{Dirichlet}(8, 4) \rightarrow$ stroke probability $\approx 67\%$.

🍷 Dirichlet-Multinomial: The Key Idea ---

The **Dirichlet** distribution is a prior over probability vectors — it represents our belief about class probabilities *before* seeing data.

$$\theta \mid \mathbf{N}_j, \alpha \sim \text{Dirichlet}(\alpha + \mathbf{N}_j)$$

In plain English: start with a prior guess (α), then add observed counts ($\mathbf{N}_j$) — the posterior is just the sum. No complex integrals needed.

	Prior	Observed	Posterior
Stroke	$\alpha_1 = 1$	$N_1 = 7$	8
No Stroke	$\alpha_2 = 1$	$N_2 = 3$	4

Estimated stroke probability: $8/(8 + 4) \approx 67\%$

Generative Model ---

- Sample a decision list length $m \sim p(m|\lambda)$.
- Sample the default rule parameter $\theta_0 \sim \text{Dirichlet}(\alpha)$.
- For decision list rule $j = 1, \dots, m$:
 - Sample the cardinality of antecedent a_j in d as $c_j \sim p(c_j|c_{<j}, \mathcal{A}, \eta)$.
 - Sample a_j of cardinality c_j from $p(a_j|a_{<j}, c_j, \mathcal{A})$.
 - Sample rule consequent parameter $\theta_j \sim \text{Dirichlet}(\alpha)$.
- For observation $i = 1, \dots, n$:
 - Find the antecedent a_j in d that is the first that applies to x_i .
 - If no antecedents in d apply, set $j = 0$.
 - Sample $y_i \sim \text{Multinomial}(\theta_j)$.

$$p(d \mid \mathbf{x}, \mathbf{y}, \mathcal{A}, \alpha, \lambda, \eta) \propto \underbrace{p(\mathbf{y} \mid \mathbf{x}, d, \alpha)}_{\text{likelihood}} \cdot \underbrace{p(d \mid \mathcal{A}, \lambda, \eta)}_{\text{prior}}$$

In plain English: a list d is good if it both explains the data well *and* was a plausible list to begin with. Bayes combines both signals.

Prior: Encouraging Sparsity ---

$$p(d \mid \mathcal{A}, \lambda, \eta) = p(m \mid \mathcal{A}, \lambda) \prod_{j=1}^m p(c_j \mid c_{<j}, \mathcal{A}, \eta) p(a_j \mid a_{<j}, c_j, \mathcal{A})$$

In plain English: the prior penalizes long lists (λ) and rules with many conditions (η) — it's a built-in preference for simplicity.

- λ : expected number of rules — lower $\lambda \rightarrow$ shorter lists preferred
- η : expected number of conditions per rule — lower $\eta \rightarrow$ simpler rules preferred
- Both use **truncated Poisson** so the list length can never exceed the number of available rules

🍷 Likelihood ---

$$p(\mathbf{y} \mid \mathbf{x}, d, \alpha) = \prod_j \int \text{Multinomial}(\mathbf{N}_j \mid \boldsymbol{\theta}_j) \text{Dirichlet}(\boldsymbol{\theta}_j \mid \alpha) d\boldsymbol{\theta}_j$$

In plain English: for each rule in the list, count how many observations of each class it captures — then ask how probable those counts are. Multiply across all rules to get the total likelihood.

- Thanks to Dirichlet-Multinomial conjugacy, $\boldsymbol{\theta}_j$ can be integrated out analytically — no numerical approximation needed
- A list scores high when each rule captures observations that are mostly from the same class

MCMC: Searching Over Decision Lists ---

BRL uses **Markov Chain Monte Carlo** to sample lists from the posterior — instead of finding one optimal list, it explores many.

At each iteration, propose a new list d^* from the current d^t via one of three moves:

- **Add:** insert a new rule from $\mathcal{A}$
- **Remove:** delete a rule from d^t
- **Move:** relocate an existing rule to a different position

In plain English: think of it as randomly editing the current list and deciding whether the new version is better — good edits are usually accepted, bad ones are accepted with small probability to avoid getting stuck.

MCMC: Metropolis-Hastings ---

$$A = \min \left(1, \frac{p(d^* \mid \mathbf{x}, \mathbf{y}, \mathcal{A}) \cdot Q(d^t \mid d^*)}{p(d^t \mid \mathbf{x}, \mathbf{y}, \mathcal{A}) \cdot Q(d^* \mid d^t)} \right)$$

In plain English: compare how good the proposed list d^* is versus the current list d^t . If d^* is better, always accept it. If it's worse, accept it anyway with some probability — this prevents getting trapped in local optima.

- **Posterior ratio:** is d^* a better explanation of the data?
- **Proposal ratio:** corrects for asymmetric moves (some edits are easier to propose than others)

Implementing BRL in Python ---

```
A = fp_growth(data, min_support=0.1, max_cardinality=2) # Step 1: Mine candidate rules
d = sample_from_prior(A, lambda_=3, eta=1) # Step 2: Initialize a random decision list from the prior

for iteration in range(n_iterations): # Step 3: MCMC - sample from posterior
    # Propose a new list by randomly choosing one of three moves:
    move = random.choice(["move", "add", "remove"])

    if move == "move": d_proposed = move_rule(d) # move a rule to a different position
    elif move == "add": d_proposed = add_rule(d, A) # add a new rule from A
    else: d_proposed = remove_rule(d) # remove a rule from d

    # Compute acceptance probability (posterior ratio)
    likelihood_current = compute_likelihood(d, X, y, alpha) # uses N_j and alpha directly
    likelihood_proposed = compute_likelihood(d_proposed, X, y, alpha)
    prior_current = compute_prior(d, A, lambda_, eta)
    prior_proposed = compute_prior(d_proposed, A, lambda_, eta)

    acceptance = min(1, (likelihood_proposed * prior_proposed) /
                     (likelihood_current * prior_current))

    d = d_proposed if random.uniform(0, 1) < acceptance else d # Accept or reject
    posterior_samples.append(d)

# Step 4: Extract point estimate
d_hat = brl_point(posterior_samples)
```

Prediction ---

Once MCMC converges to a point estimate $\hat{d}$, predicting a new observation $\tilde{x}$ is straightforward:

$$p(\tilde{y} = l \mid \tilde{x}, \hat{d}, \mathbf{x}, \mathbf{y}, \boldsymbol{\alpha}) = \frac{\alpha_l + N_{j(\hat{d}, \tilde{x}), l}}{\sum_{k=1}^L (\alpha_k + N_{j(\hat{d}, \tilde{x}), k})}$$

In plain English: find the first rule in $\hat{d}$ that matches $\tilde{x}$, then report the class probabilities estimated from that rule's data — prior counts plus observed counts, normalized to sum to 1.

```
def predict(x, d, N, alpha):  
    j = first_matching_rule(x, d)  
    counts = alpha + N[j]  
    return counts / counts.sum()
```


Experiment: Stroke Prediction ---

	AUC	Training time (mins)
BRL-point	0.756 (0.007)	21.48 (6.78)
CHADS ₂	0.721 (0.014)	no training
CHA ₂ DS ₂ -VASc	0.677 (0.007)	no training
CART	0.704 (0.010)	12.62 (0.09)
C5.0	0.704 (0.011)	2.56 (0.27)
ℓ_1 logistic regression	0.767 (0.010)	0.05 (0.00)
SVM	0.753 (0.014)	302.89 (8.28)
Random forests	0.774 (0.013)	698.56 (59.66)

- Dataset: $N = 12,586$ patients, 14% stroke — **6000 $\times$ larger** than the data used to build CHADS₂
- BRL matches or exceeds CHADS₂ AUC while producing a **human-readable list**
- Note: AUC may be misleading given class imbalance — AUPRC would be more appropriate here

Interpretable Decision Sets: A Joint Framework for Description and Prediction

Himabindu Lakkaraju
Stanford University
himalv@cs.stanford.edu

Stephen H. Bach
Stanford University
bach@cs.stanford.edu

Jure Leskovec
Stanford University
jure@cs.stanford.edu

ABSTRACT

One of the most important obstacles to deploying predictive models is the fact that humans do not understand and trust them. Knowing which variables are important in a model's prediction and how they are combined can be very powerful in helping people understand and trust automatic decision making systems.

Here we propose interpretable decision sets, a framework for building predictive models that are highly accurate, yet also highly

predicted for different data points [44]. When there are multiple classes to predict, it is also important to characterize all the classes, not just the common ones.

Interpretable models are needed in many domains because they bridge the gap between domain experts and data scientists. When domain experts need to make important decisions, learning from data can improve their results, but this requires the human to understand and trust the model. From medical diagnosis to decision

(Lakkaraju, Bach, and Leskovec 2016)

Contributions ---

- A framework called **Interpretable Decision Sets (IDS)** for multi-class classification
- **Joint objective function** that simultaneously optimizes accuracy and interpretability — with a proof of **submodularity**
- Optimization via **Smooth Local Search** with a formal $2/5$ approximation guarantee
- **Quantitative interpretability metrics** + user study validation

Decision Lists vs Decision Sets ---

Decision Lists (BRL)

- Ordered `if-then-else` rules
- Order matters: first match wins
- Zero overlap by construction
- Harder for humans to reason about:
understanding one rule requires knowing
all preceding rules

Decision Sets (IDS)

- Unordered `if-then` rules
- Each rule is independent
- Overlap is minimized but allowed
- Easier to reason about: each rule stands
on its own

Problem Setup ---

Input:

- Data $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, class labels $\mathcal{C}$
- Frequent itemsets $\mathcal{S}$ mined via Apriori — e.g., **Gender = Female AND Age > 35**

Output: a decision set $\mathcal{R} \subseteq \mathcal{S} \times \mathcal{C}$

In plain English: pick a subset of (condition, class) pairs from all possible combinations — the result is your rule set.

$$\operatorname{argmax}_{\mathcal{R} \subseteq \mathcal{S} \times \mathcal{C}} \sum_{i=1}^7 \lambda_i f_i(\mathcal{R})$$

In plain English: find the rule set $\mathcal{R}$ that maximizes a weighted sum of seven desirable properties.

🍷 Desiderata: What Makes a Good Rule Set? ---

Property	What it means
Parsimony	Few rules, short conditions
Distinctness	Rules don't cover the same data points
Class Coverage	At least one rule per class
Precision	Rules don't misclassify the points they cover
Recall	Rules collectively cover all data points

Each property is captured by one or more terms f_i in the objective — and the λ_i weights let you trade them off.

🍷 Objective: Parsimony & Distinctness ---

Fewer rules (f_1) and shorter rules (f_2)

$$f_1(\mathcal{R}) = |\mathcal{S}| - \text{size}(\mathcal{R}), \quad f_2(\mathcal{R}) = L_{\max} \cdot |\mathcal{S}| - \sum_{r \in \mathcal{R}} \text{length}(r)$$

In plain English: prefer 12 rules over 30, and prefer `Age > 60` over `Age > 60 AND Diabetes AND Hypertension AND Smoker`.

Low overlap (f_3, f_4)

- f_3 : two rules predicting the **same** class shouldn't cover the same patients
- f_4 : two rules predicting **different** classes shouldn't cover the same patients — otherwise a tie-breaking rule decides, which hurts interpretability

🍷 Objective: Coverage, Precision & Recall ---

Class coverage (f_5)

In plain English: every class must have at least one rule — this is critical for rare but important classes like rare diseases.

Precision (f_6) and Recall (f_7)

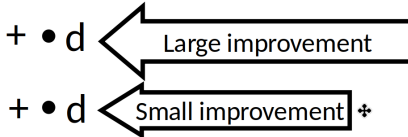
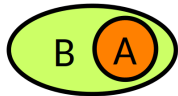
$$f_6(\mathcal{R}) = N \cdot |\mathcal{S}| - \sum_{r \in \mathcal{R}} |\text{incorrect-cover}(r)|$$

In plain English (f_6): each rule should cover mostly patients of the right class — not a mix.

In plain English (f_7): the rule set should correctly classify as many patients as possible — leaving few uncovered.

🍷 Submodularity: Why It Matters ---

$$\underbrace{F(A \cup d) - F(A)}_{\text{Gain of adding } d \text{ to a small set}} \geq \underbrace{F(B \cup d) - F(B)}_{\text{Gain of adding } d \text{ to a large set}}$$



In plain English: adding a rule to a small set helps a lot; adding the same rule to a large set helps less — this is **diminishing returns**, and it's exactly what submodularity captures for set functions.

Key Property

A non-negative linear combination of submodular functions is submodular — so the full IDS objective inherits this property.

Optimization: Smooth Local Search ---

Maximizing the IDS objective is **NP-hard** — the space of possible rule sets is exponential in $|\mathcal{S}|$.

Smooth Local Search (SLS) provides a $2/5$ approximation guarantee:

- 1 Start with $\mathcal{R} = \emptyset$
- 2 Estimate marginal contribution of each candidate rule
- 3 Add rules that improve the objective enough
- 4 Remove rules that hurt the objective enough
- 5 Repeat until convergence; return a random subset (the “smoothing” step)

In plain English: instead of checking all $2^{|\mathcal{S}|}$ possible sets, SLS makes smart local edits — and guarantees the result is worth at least $2/5$ of the theoretical optimum.

Evaluation: Datasets ---

Dataset	# Datapoints	Features	Classes
Bail Outcomes	86K	Gender, age, offense details, criminal record	No Risk, Failure to Appear, New Criminal Activity
Student Performance	21K	Gender, age, grades, absence rates, suspension history	Graduated on Time, Delayed Graduation, Dropped Out
Medical Diagnosis	150K	Ailments, age, BMI, gender, smoking, medical history	Asthma, Diabetes, Depression, Lung Cancer, Rare Blood Cancer

All three datasets involve **high-stakes decisions** — justice, education, and medicine — where interpretability is not optional.

🍷 Results: Predictive Performance ---

Method	Bail	Student	Medical
IDS	69.78	75.12	61.19
Bayesian Decision Lists	67.18	72.54	59.18
Decision Trees	70.08	75.31	63.28
Gradient Boosted Trees	71.23	77.18	64.21
Random Forests	70.87	77.12	63.92

IDS trades a small accuracy gap against black-box models for **full interpretability**. Note: class imbalance is likely in all three datasets — AUC results should be interpreted with caution.

🍷 Results: Interpretability Metrics ---

Method	Frac. Overlap	Frac. Uncovered	Avg. Rule Length	Num. Rules	Frac. Classes
IDS	0.09	0.13	3.17	12	1.00
BDL	0.00	0.18	8.46	11	0.67
CBA	0.00	0.14	8.60	32	1.00
CN2	0.12	0.14	9.78	38	1.00

IDS rules are **3× shorter** than baselines, cover **all classes** (BDL misses 2 rare diseases), and use far fewer rules than CBA and CN2.

🍷 Results: User Study ---

47 Stanford students assigned to either IDS or BDL — never both. Each answered 10 objective + 2 descriptive questions about a medical diagnosis rule set.

Task	Metric	IDS	BDL
Descriptive	Accuracy	0.81	0.17
	Avg. Time (secs.)	113	397
Objective	Accuracy	0.97	0.82
	Avg. Time (secs.)	28	36

IDS users were **4.7× more accurate** on descriptive tasks and **3.5× faster** — confirming that unordered rules are fundamentally easier for humans to reason about.

BRL vs IDS -- When to Use Each? ---

	BRL	IDS
Structure	Ordered if-then-else list	Unordered if-then rules
Prediction	Probabilistic (credible intervals)	Deterministic (precision-based)
Best for	Binary classification, medical risk scoring	Multi-class problems with rare classes
Uncertainty	Built-in via Dirichlet-Multinomial	Not provided
Optimization	MCMC sampling	Submodular optimization (2/5 guarantee)
Rule overlap	Zero by construction	Minimized but allowed
Use when	You need calibrated probabilities	You need equally good rules for all classes

🍷 Key Takeaways ---

- Interpretability is **multifaceted** — sparsity, overlap, coverage, and rule length all matter independently
- **Ordered vs. unordered rules** is not just a structural choice — it fundamentally affects how humans understand models (user study: $4.7\times$ accuracy gap)
- Both papers show interpretability and accuracy are **not mutually exclusive**
- Evaluating interpretability requires **both quantitative metrics and user studies**
- The right model depends on the task: BRL excels at **uncertainty quantification**, IDS at **multi-class clarity**

Thank You!

- Lakkaraju, Himabindu, Stephen H. Bach, and Jure Leskovec. 2016. “Interpretable Decision Sets: A Joint Framework for Description and Prediction.” In *Proceedings of the 22nd Acm Sigkdd International Conference on Knowledge Discovery and Data Mining*, 1675–84. ACM.
- Letham, Benjamin, Cynthia Rudin, Tyler H. McCormick, and David Madigan. 2015. “Interpretable Classifiers Using Rules and Bayesian Analysis: Building a Better Stroke Prediction Model.” *The Annals of Applied Statistics* 9 (3): 1350–71.